We could try to create margins along the height and width of the layout as we did earlier. Or we could provide information about each node as a information box that opens when we click it, as with the soccer data. But a better option is to give the user the ability to drag the canvas up and down and left and right to see more of the visualization.

To do this, we use the D3 zoom behavior, `d3.behavior.zoom`, which creates a set of event listeners. A behavior is like a component, but instead of creating graphical objects, it creates events (in this case for drag, mousewheel, and double-click) and ties those events to the element that calls the behavior. With each of these events, a zoom object changes its `.translate()` and/or `.scale()` values to correspond to the traditional dragging and zooming interaction. You'll use these changed values to adjust the position of graphical elements in response to user interaction. Like a component, the zoom behavior needs to be called by the element to which you want these events attached. Typically, you call the zoom from the base `<svg>` element, because then it fires whenever you click anything in your graphical area. When creating the zoom component, you need to define what functions are called on *zoomstart*, *zoom*, and *zoomend*, which correspond (as you might imagine) to the beginning of a zoom event, the event itself, and the end of the event, respectively. Because zoom fires continuously as a user drags the mouse, you may want resource-intensive functions only at the beginning or end of the zoom event. You'll see more complicated zoom strategies, as well as the use of scale, in chapter 7 when we look at geospatial mapping, which uses zooming extensively.

As with other components, to start a zoom component you create a new instance and set any attributes of it you may need. In our case, we only want the default zoom component, with the `zoom` event triggering a new function, `zoomed()`. This function changes the position of the `<g>` element that holds our chart and allows the user to drag it around:

```
treeZoom = d3.behavior.zoom();
treeZoom.on("zoom", zoomed);
d3.select("svg").call(treeZoom);

function zoomed() {
  var zoomTranslate = treeZoom.translate();
  d3.select("g.treeG").attr("transform",
       "translate("+zoomTranslate[0]+","+zoomTranslate[1]+")")
};
```

Creates a new zoom component

Calls our zoom component with the SVG canvas

Keys the "zoom" event to the zoomed() function

Transform attribute changes to reflect the zoom behavior

Updating the <g> to set it to the same translate setting of the zoom component updates the position of the <g> and all its child elements.

Now we can drag and pan our entire chart left and right and up and down. In figure 5.16, we can finally read the text of the tweets by dragging the chart to the left. The ability to zoom and pan gives you powerful interactivity to enhance your charts. It may seem odd that you learned how to use something called *zoom* and haven't even dealt with